Kinect Lite: Depth Denoising

Overview

This project cleans noisy depth images from a Kinect-style camera with spatial filters, and measures how much each filter helps. On simulated data with a known true depth, the best-scoring filters cut the average depth error roughly in half (26.2 mm to 12.8 mm), mainly by removing large spikes at object edges.

The work runs in five steps:

1. **Sense:** the camera gives one depth value per pixel, in millimetres, with some pixels missing.
2. **Lift to 3D:** each pixel becomes a 3D point using the camera model, giving a point cloud.
3. **Check the data:** compare the noisy depth with the true depth to see what the noise really looks like.
4. **Filter:** run each filter on single frames, with settings tuned on one set of frames.
5. **Measure:** score the filters on a different set of frames, separately for flat surfaces and for object edges.

The sections follow that order: sensor and image format, then the 3D model, the data, the noise, the filters, how they are scored, the experiment files, and finally the results and what they mean. Scope is spatial filtering of single frames only: no time filtering and no 3D registration.

How a depth camera works

A Kinect-style camera measures distance by triangulation: it projects a known infrared dot pattern and sees how far each dot shifts sideways in its infrared camera. That shift is the disparity d, and depth follows from it:

```
Z = \frac{f \, b}{d}
```

Here f is the focal length in pixels and b the baseline between projector and camera. Because Z goes as 1/d and d is measured in steps, depth precision gets worse roughly with the square of the distance: a few millimetres at 1 m, several centimetres at 4 m.

What the image file holds. Both datasets store each frame as a 640 x 480 PNG with one 16-bit integer per pixel. The value divided by 5000 gives metres, so 5000 means 1 m and the step is 0.2 mm. A value of 0 means no measurement.

Why pixels go missing. The sensor returns 0 where it sees no dot: surfaces too close (below about 0.4-0.5 m) or too far, shiny or very dark surfaces, and areas hidden from the projector behind an object (shadows beside edges).

In the code every depth image is converted to metres as float numbers, with missing pixels stored as NaN ("not a number").

From depth image to 3D point cloud

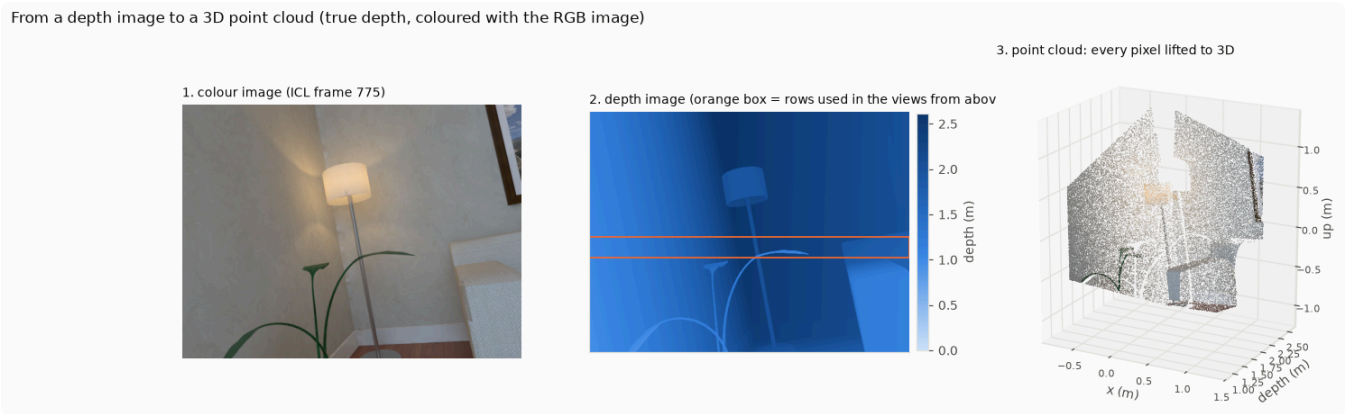
Every depth pixel is a 3D point: the pinhole camera model turns pixel position (u, v) and depth Z into coordinates (X, Y, Z) in metres. Doing this for all pixels gives a point cloud, a 3D picture of the scene from one frame.

```
X = \frac{(u - c_x)\,Z}{f_x}, \qquad Y = \frac{(v - c_y)\,Z}{f_y}, \qquad Z = Z
```

The four camera numbers are the intrinsics: focal lengths fx, fy (in pixels) and the image centre cx, cy.

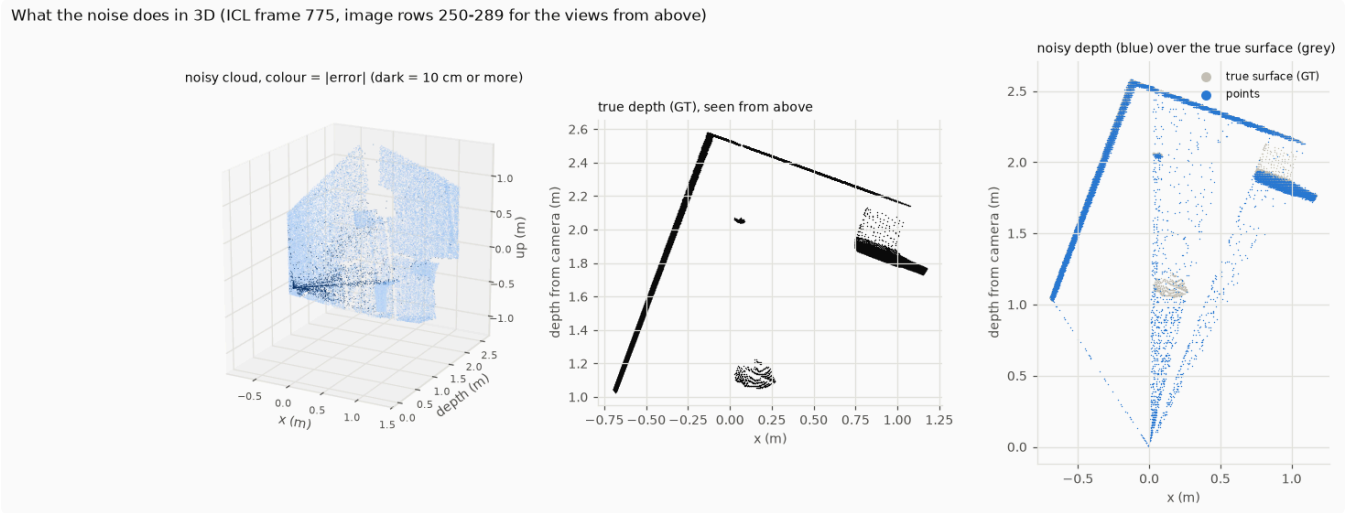
dataset	fx	fy	cx	cy
ICL-NUIM	481.2	480.0	319.5	239.5
TUM fr3	535.4	539.2	320.1	247.6

ICL-NUIM lists fy as -480 because it counts Y upwards; the shape of the cloud is the same.



The picture above shows one ICL frame: the colour image, the depth image (lighter = nearer), and the same pixels lifted to 3D. The walls meet at a corner, and the lamp, plant and sofa stand in front of them.

Why the 3D view matters for denoising. A wrong depth value moves its point along the camera's line of sight, toward or away from the camera. Seen from above, a clean surface is a thin line; noise shows as points scattered off that line. The next picture shows a 40-row band of the image seen from above.



The noisy points (blue, right) are not scattered at random. They lie on straight lines running from the camera at (0, 0) to the edges of the lamp and the sofa: each is a depth value that is too small, on the right line of sight. Section 5 explains where these come from.

An interactive version, which you can rotate and zoom, is in `outputs/pointcloud_viewer_icl.html` (true, noisy and filtered clouds; click a name to show or hide it).

The two datasets

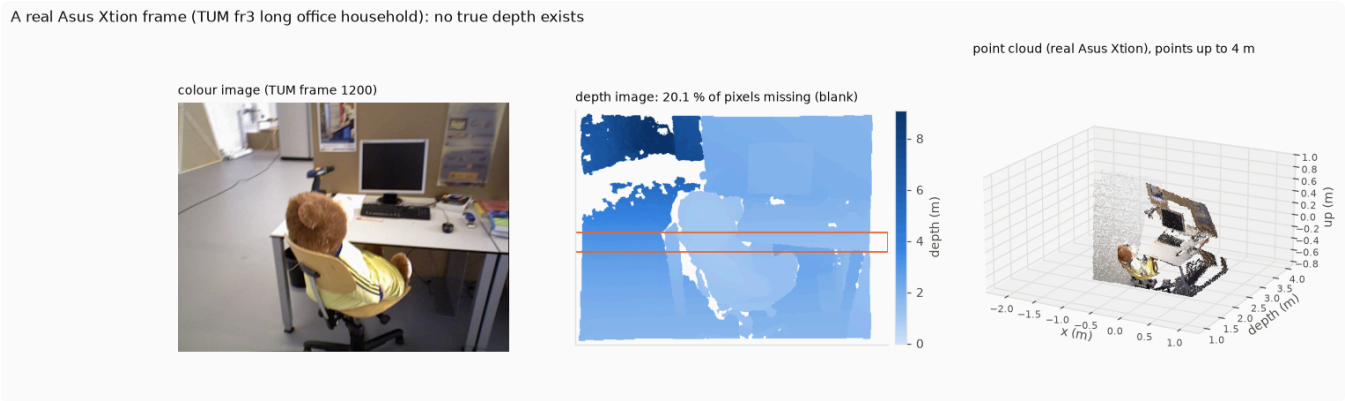
ICL-NUIM gives the scores, because it has the true depth; TUM shows what the same filters do to a real Kinect-type camera (an Asus Xtion), where no true depth exists.

	ICL-NUIM living room kt0	TUM fr3 long office household
Source	rendered (synthetic) living room	recorded with a real Asus Xtion (Kinect-type sensor)
Frames	1,509 (we use 61 of them)	about 2,500 (we use 50)

	ICL-NUIM living room kt0	TUM fr3 long office household
True depth	yes: a noise-free copy of every frame	no
Missing pixels	about 0.5 %	about 16.5 % on average (20 % in the frame shown below)
Noise	simulated (below)	real sensor noise
Used for	tuning and scoring the filters	checking how much the filters change real data

How ICL-NUIM simulates Kinect noise (Handa et al., ICRA 2014, Section V-A): each pixel is shifted by a small random offset (standard deviation 0.5 px) and resampled with bilinear interpolation; Gaussian noise is added to the disparity (standard deviation 1/6); then the disparity 35130 / Z is rounded to a whole number, which creates the step-like depth bands of a real Kinect. So the noisy image is a known, controlled corruption of the true one.

TUM fr3 (Sturm et al., IROS 2012) is a hand-held Asus Xtion (the same depth technology as the original Kinect) moving around two desks. It has the camera's true path (from motion capture) but not the true depth of each pixel.



The white areas in the TUM depth image are missing pixels: a large band in the upper left, and thin strips along the outlines of the chair and the teddy bear, where the projector's pattern is hidden.

What the noise looks like

On ICL, 4 % of the pixels carry 56 % of the depth error: a few values far too near the camera, and the pixels on object edges. The other 96 % are only mildly noisy (15.6 mm average error).

Depth noise comes in five kinds:

- **Random noise:** small jitter on every pixel, larger for far surfaces.
- **Quantisation:** depth comes in steps, because disparity is a whole number; far away the steps are centimetres apart.
- **Missing pixels:** no value at all (0 in the file).
- **Mixed or flying pixels:** at an edge, a pixel takes a value between the near and the far surface.
- **Outliers:** isolated values far from both surfaces.

What we measured on ICL (30 frames, noisy vs true depth):

pixel group	share of pixels	share of all error	average error
values below 0.4 m (closer than a Kinect can see)	0.5 %	25 %	1,573 mm
2-pixel band around object edges	3.5 %	31 %	311 mm
everything else	96 %	44 %	15.6 mm

The edge errors are mostly outliers, not blur. Of the edge pixels more than 5 cm off, 70 % are nearer than both surfaces at that edge, by about 1 m on average. Only a small part is classic mixing between the two surfaces. In 3D (Section 3) these values sit on the line of sight between the camera and the edge, spread evenly from 0 m to the surface.

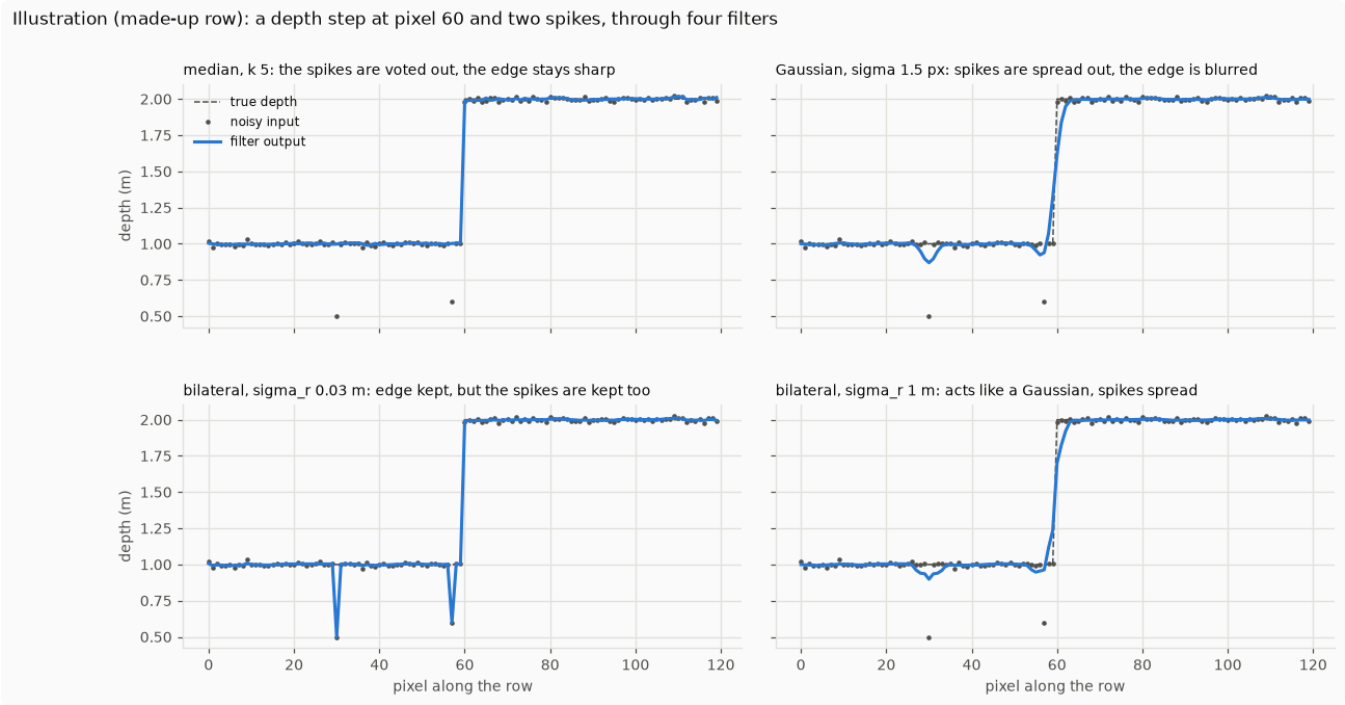
The likely cause is the simulation's bilinear resampling next to a missing (0 m) value: mixing a depth with 0 gives anything between 0 and that depth. This fits every check we ran, but the dataset authors do not state it, so we treat it as a likely explanation, not a fact.

Noisy and true images are about 1 pixel out of line. Shifting the true image by about 1.25 pixels lowers the flat-area error from 15.0 to 13.5 mm. The best shift changes across the image, so it is not a simple offset. We score against the true image as given; this adds about 1.5 mm of error to every filter alike, and more at edges.

These findings decide which filters can work: a filter that rejects outliers can remove the edge spikes; a filter that averages will spread them.

The denoising filters

Every filter here replaces each pixel's depth by a combination of the depths in a small window around it. They differ in how they combine them: medians vote, Gaussians average, bilaterals average only over similar depths.



The picture uses a made-up row (a step from 1 m to 2 m, plus two spikes) to show each filter's behaviour before any real data.

Median

The output is the middle value of the k x k window, sorted. A spike is one value among k x k, so it never reaches the middle and is voted out; an edge stays sharp, because on each side most of the window is on that side. Its weakness: near an edge it can pick the other surface's value and shift the outline by a pixel or two.

Two versions:

- **Plain median** puts missing pixels into the window as 0 m. It can fill a hole (most of the window valid) and can also delete a measured pixel (most of it missing).
- **Invalid-aware median (iamedian)** takes the median of the valid pixels only. Holes stay holes and no measured pixel is lost.

Gaussian

A weighted average where weights fall off with distance from the centre pixel:

$$\hat{Z}(p) = \frac{\sum_{q} w_s(p,q)\, Z(q)}{\sum_{q} w_s(p,q)}, \qquad w_s(p,q) = \exp\!\left(-\frac{\lVert p-q \rVert^2}{2\sigma_s^2}\right)$$

It smooths random noise well, but it averages across edges (blur) and spreads a spike over its neighbours instead of removing it.

Bilateral

The Gaussian weight times a second weight on depth difference, so neighbours at a very different depth count less:

$$w(p,q) = \exp\!\left(-\frac{\lVert p-q \rVert^2}{2\sigma_s^2}\right) \exp\!\left(-\frac{(Z(p)-Z(q))^2}{2\sigma_r^2}\right)$$

σs (pixels) sets how far it looks; σr (metres) sets how different a depth may be and still count. A small σr keeps edges, but also keeps a spike, because the spike's neighbours are "too different" to correct it. A large σr turns it into a Gaussian.

OpenCV versions vs masked versions

OpenCV's Gaussian and bilateral need a number in every pixel, so missing pixels go in as 0 m and pull their neighbours towards the camera. The **masked** versions leave missing pixels out of the sum (they divide by the weights of valid pixels only). Away from holes the two give the same result to within 0.004 mm.

Hole filling

Fills small interior holes from the edge inward, each pixel with the median of its valid neighbours. Holes larger than a set area, or touching the image border, are left alone. An edge guard skips a pixel when its neighbours span more than a set depth range, because guessing across an edge is likely wrong. Measured pixels are never changed.

The 0.4 m gate

Before filtering, every depth below 0.4 m is treated as missing. A Kinect cannot measure that close, so such values are errors. On ICL they are 0.5 % of pixels but 25 % of the error.

How the filters are measured

The main score is the mean absolute error (MAE): the average distance between filtered depth and true depth, in millimetres, reported separately for flat areas and for edges.

```
\text{MAE} = \frac{1}{N} \sum_{p} \left| \hat{Z}(p) - Z_{\text{true}}(p)
\right|
```

Edges and flat areas. An edge is where the true depth jumps by more than 5 cm (or 3 % of the depth) between neighbouring pixels. Edges come from the true image only, so a filter cannot change where they are. Each frame is split into:

- **edge band:** pixels within 2 px of an edge (about 3.5 % of pixels);
- **flat:** pixels more than 5 px from any edge;
- **all:** every scored pixel.

One number for "all" hides what happens where: a filter can fix edges and slightly worsen flat areas, or the reverse.

Same pixels for every filter. Some filters fill holes and some delete pixels. To keep the comparison fair, the main scores use only pixels that are valid in the true image, the input and every filter's output (the matched mask). A second score, "own MAE", uses each filter's own valid pixels, to show the cost of filled or deleted pixels.

Tuning and testing on different frames. Settings are tuned on frames 0, 50, 100 … (31 frames) and scored on frames 25, 75, 125 … (30 frames). Picking settings on the same frames you score would flatter the result. The two sets come from the same video, so this is a fair check but not a fully independent test.

Per-frame change. For each test frame we also record how much a filter lowers or raises the MAE compared with the raw input, and on how many frames it makes things worse.

TUM has no true depth, so it gets no error score: only how much the filter changed the depth (on flat areas and at edges), how rough flat areas are afterwards, and how many pixels are missing.

The experiments folder

Each file in `experiments/` is a recipe: which filters to run, with which settings, on which frames. One command runs a recipe and writes its results to `outputs/<recipe name>/`.

A recipe looks like this (`tune_gaussian_masked.json`):

```
{
  "name": "tune_gaussian_masked",
  "dataset": "icl",
  "frames": "0:1509:50",
  "configs": [
    {"method": "gaussian_masked",
     "grid": {"sigma_px": [0.5, 0.75, 1, 1.5, 2, 3, 5]},
     "min_depth_m": [null, 0.4]}
  ]
}
```

- `dataset`: `icl` (has true depth, gets error scores) or `tum` (real Asus Xtion, change only).
- `frames`: start:stop:step, so `0:1509:50` means frames 0, 50, 100 ... 1500.
- `method`: a filter name from `denoise/filters.py`.
- `grid`: every listed value is tried; two parameters with 3 values each give 9 settings.
- `min_depth_m`: `null` = no gate, `0.4` = the 0.4 m gate; a list runs both.

The raw input is always added as the reference. This recipe gives 7 sigmas x 2 gates + raw = 15 configs.

The recipes, in the order they were run:

file	what it does	frames
`reproduce_original.json`	the 12 settings of the earlier run, to check this code gives the same numbers (it does, exactly)	test
`tune_median.json`	plain median, k = 3 to 11	tuning
`tune_iamedian.json`	invalid-aware median, k = 3 to 13	tuning
`tune_gaussian_cv2.json`	OpenCV Gaussian, sigma 0.5 to 5 px	tuning
`tune_gaussian_masked.json`	masked Gaussian, sigma 0.5 to 5 px	tuning
`tune_bilateral_cv2.json`	OpenCV bilateral, 72 settings of d, sigma_s, sigma_r	tuning
`tune_bilateral_masked.json`	masked bilateral, same 72 settings	tuning
`tune_bilateral_*_ext.json`	larger windows and sigma_r, because the best results sat at the edge of the first grid	tuning

file	what it does	frames
`tune_hole_fill.json`	hole filling, 4 hole sizes x 4 edge guards	tuning
`eval_selected.json`	the lowest-error setting per filter family on the tuning frames, plus the original settings	test
`tum_selected.json`	the same settings on real Asus Xtion data	TUM

The last two are written by `denoise/select_configs.py`, not by hand.

What a run writes to `outputs/<name>/`:

- `summary.csv`: one row per setting, the numbers to read first;
- `per_frame.csv`: one row per frame and setting;
- `experiment_resolved.json`: the exact list of settings and the command used;
- `environment.json`: library versions.

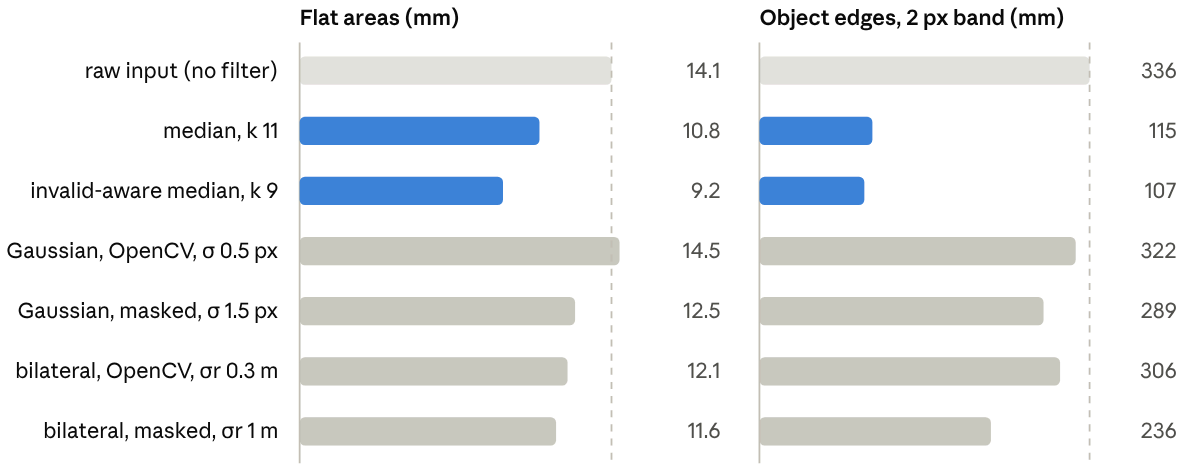
A finished run is never overwritten, so every number in this document can be traced to a file.

Results

On the ICL test frames, the two median filters roughly halve the overall error (26.2 mm to 12.8-14.8 mm) and remove about two-thirds of the edge error. Gaussian and bilateral filters improve flat areas less and edges much less.

Medians remove two-thirds of the edge error; averaging filters 4-30 %

Mean absolute error, 30 ICL test frames, one tuned setting per filter, 0.4 m gate. Blue = median filters.

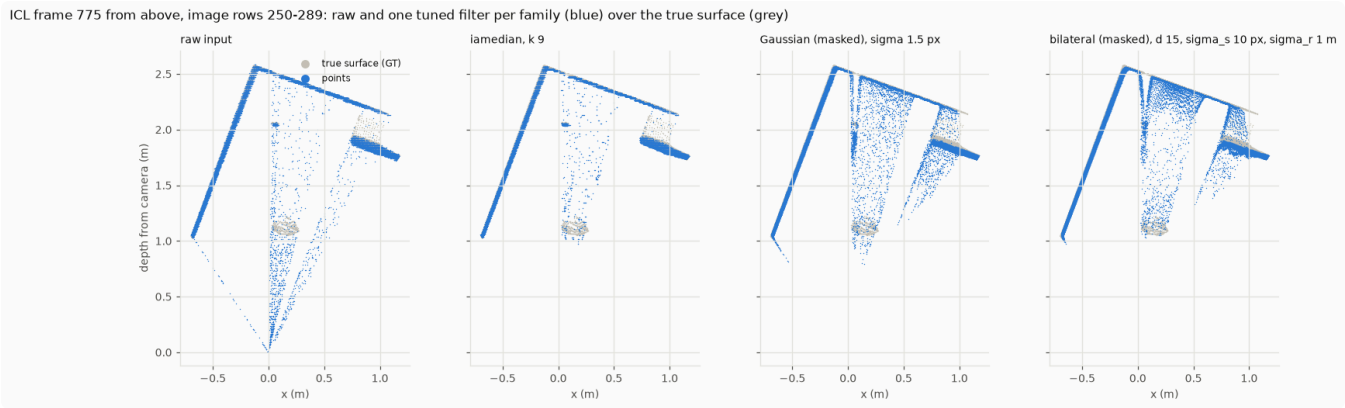


Dashed lines = raw input. Bars start at 0 mm.

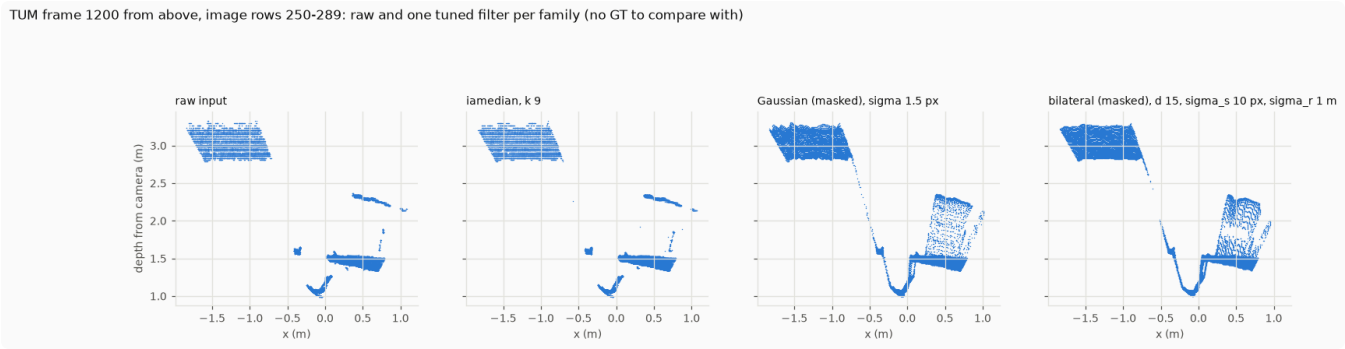
outputs/eval_selected/summary.csv · ICL frames 25:1509:50 (30 frames)

Every filter in the chart gave a lower overall error than the raw input on all 30 test frames, except the OpenCV Gaussian, which was worse than raw on 12 of 30. The improvements seen on the tuning frames held on the test frames to within 6 percentage points.

The same thing in 3D. One frame from above, raw input and three filters (blue) over the true surface (grey). The median removes the lines of points in front of the lamp and sofa; the Gaussian and bilateral turn them into fans behind the edges.



Real Asus Xtion (TUM), no true depth. The same filters, the same view:



On real data the median keeps the gaps between surfaces clean. The Gaussian and the ICL-tuned bilateral (σr = 1 m) add new points that bridge the desk (1.5 m) and the wall behind it (2.3 m): these are flying pixels created by the filter.

TUM, 50 frames	depth change on flat areas (mm)	depth change at edges (mm)	flat roughness (mm)
raw input	0	0	3.06
median, k 11	5.0	50.4	2.45
invalid-aware median, k 9	3.5	34.2	2.21
Gaussian, OpenCV, σ 0.5 px	6.5	29.8	2.74
Gaussian, masked, σ 1.5 px	4.3	79.1	0.38
bilateral, OpenCV, σr 0.3 m	7.0	42.6	0.27
bilateral, masked, σr 1 m	8.4	87.2	0.27

On TUM these are changes, not errors: smoother flat areas (lower roughness) are visible, but whether a change at an edge is a fix or damage cannot be told without true depth. The 0.4 m gate has no effect on TUM, which has no depth below 0.4 m.

What the results mean, and the limits

The kind of noise decides which filter works: spikes need a filter that votes them out (median), random jitter needs one that averages (Gaussian, bilateral). ICL's error is dominated by spikes, so medians score far better there.

What each filter family is good and bad at

filter	good at	bad at
median / invalid-aware median	removing spikes and outliers; keeping edges sharp	moves outlines by 1-2 px (wrong-side edge pixels 8.6 % to 16-21 %); slowest of the simple filters
Gaussian (masked)	smoothing flat surfaces; very fast (12 ms per frame)	spreads spikes; blurs edges and creates flying pixels between surfaces
Gaussian (OpenCV)	fast (4 ms)	worse than raw on flat areas: holes enter as 0 m and pull depth toward the camera
bilateral	smoothing while respecting edges, if σr is small	small σr keeps spikes; large σr behaves like a Gaussian; masked version is slow in our code (1.8 s per frame)
hole filling	closing small holes (TUM has 16.5 % missing)	guesses can be wrong at edges; the edge guard cuts wrong fills sharply

Handling of missing pixels matters as much as the filter. The same Gaussian is worse than raw when holes count as 0 m, and better than raw when they are left out.

The 0.4 m gate helps averaging filters most on ICL (masked bilateral: -4.9 mm vs -2.7 mm without it). It does nothing on TUM, which has no depth below 0.4 m.

Limits

- ICL's noise is simulated. Its edge spikes look like a side effect of the simulation, so the large median advantage at edges may be smaller on a real camera.
- The noisy and true ICL images are about 1 px out of line; this adds about 1.5 mm to every flat-area score.

- Tuning and test frames come from the same video, 25 frames apart.
- Settings were picked by overall error, which edges dominate. Picking by flat-area error would choose other settings; all tested settings are in `outputs/tune_*/summary.csv`.
- TUM has no true depth, so on real data we can show what changes, but not whether it is right.

Where everything is

Start with the `RESULTS/` folder: numbered pictures in reading order, one table, two 3D viewers, and a README that says what each shows.

folder or file	contents
`RESULTS/`	the results to look at first (rebuilt by `denoise/make_results.py`)
`denoise/`	all code; `README.md` at the top level lists what each file does
`experiments/`	the run recipes (Section 8), with their own `README.md`
`outputs/<run>/`	raw numbers of every run (`summary.csv`, `per_frame.csv`)
`outputs/figures/`	every figure, including all tuning curves
`docs/`	findings, data audit, experiment plan, references
`denoise_code.zip`	code, recipes and docs in one file to share (no datasets)

Run everything again, in this order (Windows, from the project folder):

1. `.venv\Scripts\python.exe denoise\audit.py` and `denoise\audit2.py`: the data checks.
2. `sh denoise/run_tuning.sh`: every tuning recipe (the masked bilateral takes about an hour).
3. `.venv\Scripts\python.exe denoise\select_configs.py`: pick a setting per filter family.

4. `.venv\Scripts\python.exe denoise\run.py experiments\eval_selected.json --save-frames 275,775,1225`, then the same with `tum_selected.json`.
5. `denoise\plots.py tuning`, `eval`, `visuals`; then `denoise\pointcloud.py` and `denoise\illustrate.py` for the pictures.
6. `denoise\make_results.py`: rebuild `RESULTS/`.

The datasets are not included: ICL-NUIM goes in `ICL NUIM/` and TUM fr3 in `RGB TUM/`, as listed in `README.md`.